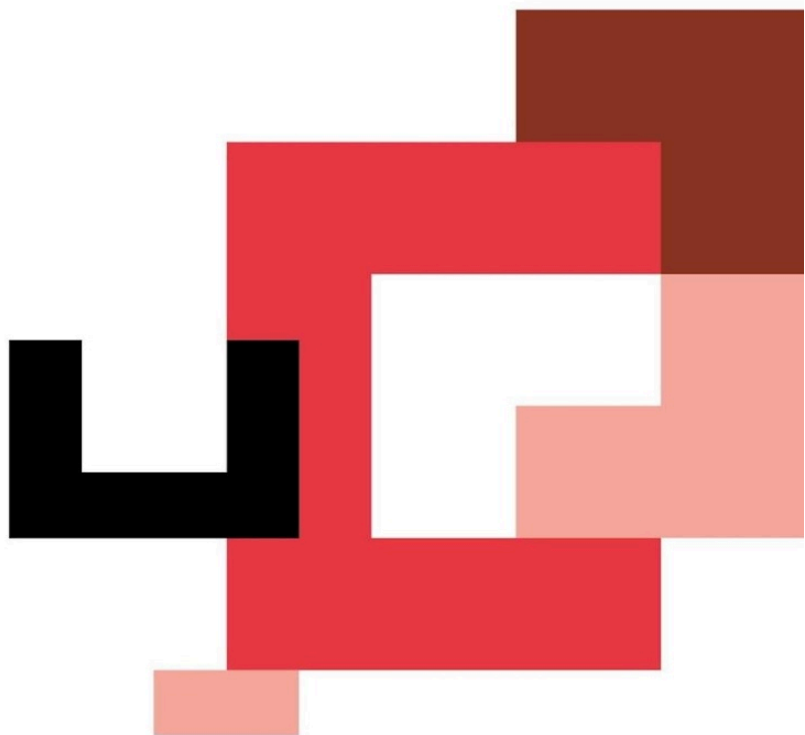




FULL STACK DEVELOPER + IA

Ejemplo REDUX



Objetivo

- Entender estado global
 - Crear un store
 - Crear un slice
 - Usar useSelector y useDispatch
-

Paso 0 — Crear proyecto

Entra en:

<https://stackblitz.com>

Selecciona:

React

Paso 1 — Instalar Redux

En la terminal:

```
Shell  
npm install @reduxjs/toolkit react-redux
```

Paso 2 — Estructura

Crear:

src/

store/

└─ store.js

features/

└─ counter/

└─ counterSlice.js

App.jsx

main.jsx

Paso 3 – Crear el slice

Archivo:

src/features/counter/counterSlice.js

JavaScript

```
import { createSlice } from "@reduxjs/toolkit";

const counterSlice = createSlice({
  name: "counter",
  initialState: { value: 0 },
  reducers: {

    increment: (state) => {
      state.value += 1;
    },

    decrement: (state) => {
      state.value -= 1;
    }

  }
});

export const { increment, decrement } = counterSlice.actions;
```

```
export default counterSlice.reducer;
```

Paso 4 — Crear el store

Archivo:
src/store/store.js

```
JavaScript
import { configureStore } from "@reduxjs/toolkit";

import counterReducer from "../features/counter/counterSlice";

export const store = configureStore({
  reducer: {
    counter: counterReducer
  }
});
```

Paso 5 — Conectar Redux

Archivo:
src/main.jsx

```
None
import React from "react";

import ReactDOM from "react-dom/client";

import { Provider } from "react-redux";
```

```
import { store } from "../store/store";

import App from "../App";

ReactDOM.createRoot(document.getElementById("root")).render(

  <Provider store={store}>

    <App />

  </Provider>

);
```

Paso 6 — Usar Redux en App

Archivo:
src/App.jsx

```
None
import { useSelector, useDispatch } from "react-redux";

import { increment, decrement } from
"./features/counter/counterSlice";

export default function App() {

  const count = useSelector(state => state.counter.value);

  const dispatch = useDispatch();

  return (
```

```
<div>

  <h1>Contador: {count}</h1>

  <button onClick={() => dispatch(increment())}>

    +

  </button>

  <button onClick={() => dispatch(decrement())}>

    -

  </button>

</div>

);

}
```

Resultado esperado

- Contador empieza en 0
- Botón + suma
- Botón - resta

Qué está pasando

1. El usuario hace click
2. dispatch ejecuta una acción
3. el reducer actualiza el estado
4. React re-renderiza
5. se actualiza la UI

Flujo

Usuario



dispatch(increment)



slice (reducer)



store actualiza estado



useSelector detecta cambio



re-render

Este ejemplo es el mínimo necesario para entender Redux en un entorno real.